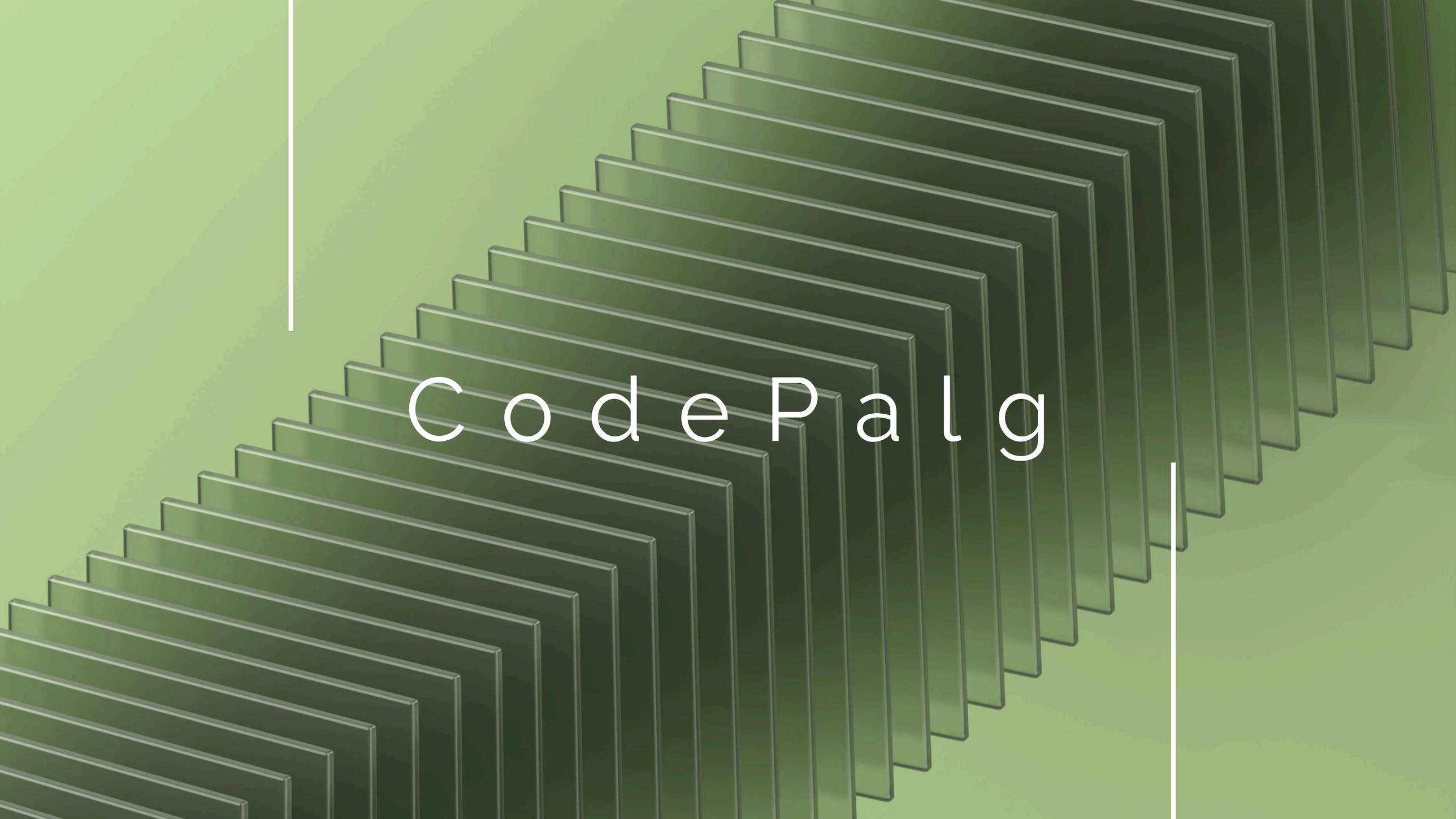

CodePalg



Selección del Dataset

1 Fuente

- IR-Plag-Dataset.zip encontrado en el OneDrive de datasets de la clase
- Tamaño: 467 archivos .java en total.

2 Organización

- original/: la solución de referencia, 1 por caso.
- plagiarized/: variantes de plagio en 6 niveles de dificultad, L1 a L6 (entre 40 y 54 archivos por caso).
- non-plagiarized/: 15 soluciones independientes por caso.

3 Propósito

Evaluar si el sistema distingue plagio de trabajo original, y con qué nivel de ofuscación deja de detectarlo.

Razones de la selección

Relevancia

- Es un dataset construido específicamente para detección de plagio en código, con etiquetas claras.

Niveles de dificultad

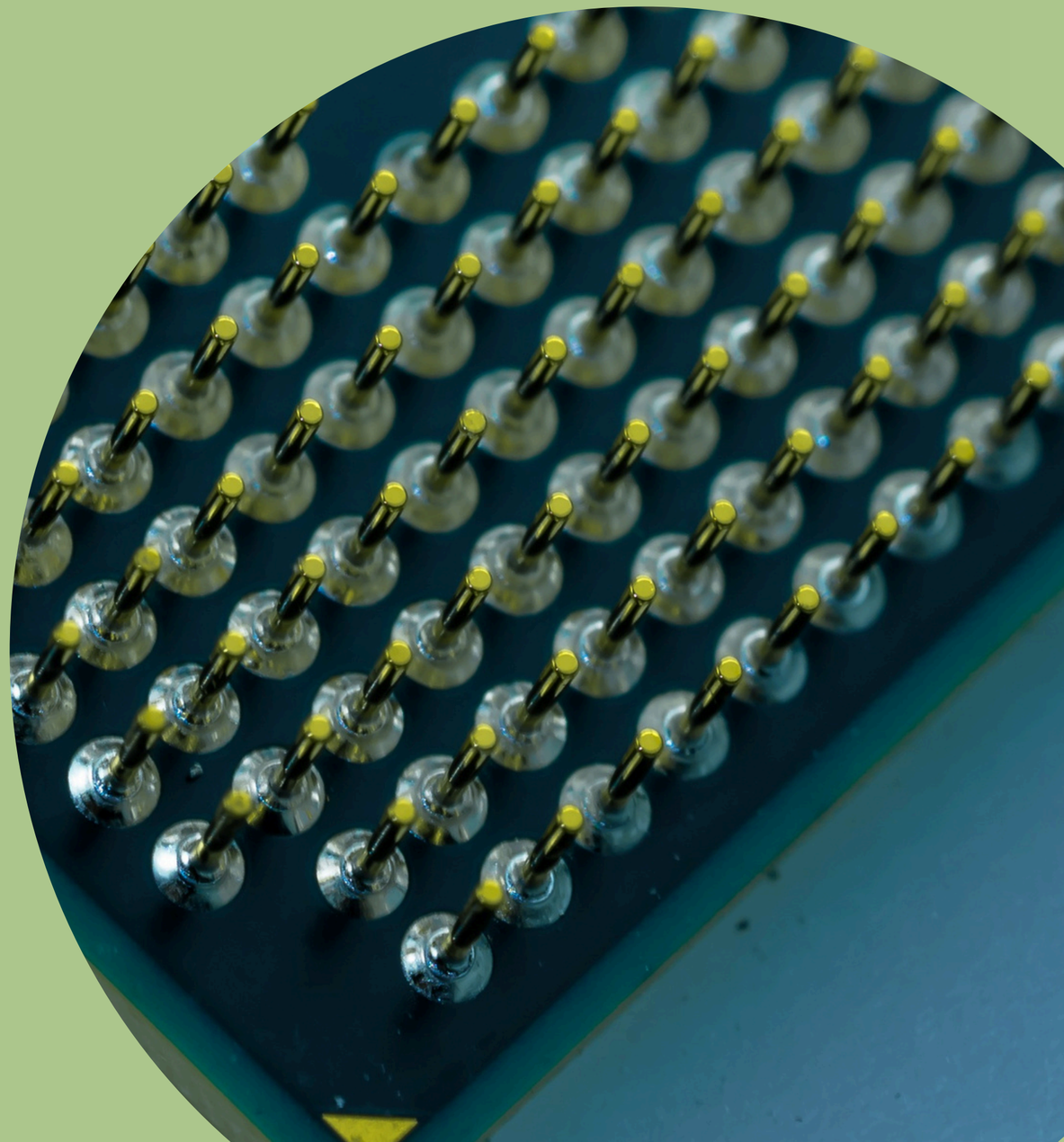
- El AST con normalización debería rendir muy bien en L1 y L2, que son cambios de nombres y formato.
- El modelo neuronal debería aportar más en L5 y L6, que son reescrituras con la misma lógica.

Aplicabilidad al modelo

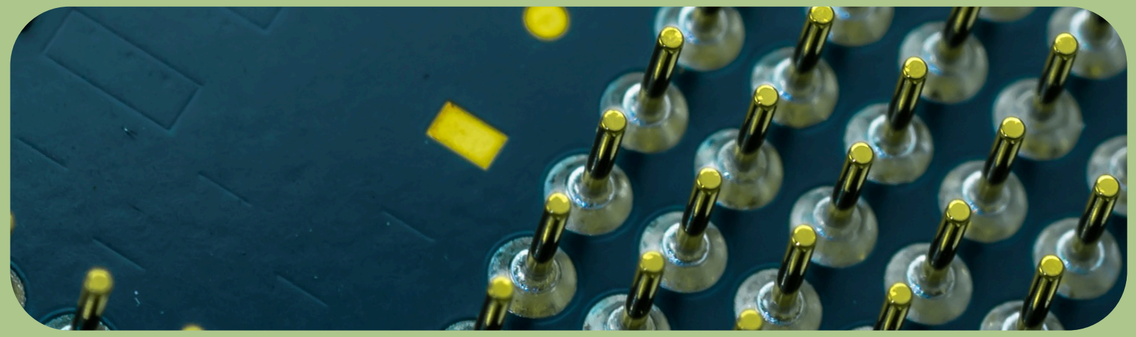
- Los pares original/candidato encajan con una red siamesa.
- Java está soportado por tree-sitter, así que el AST se puede construir sin cambios.
- Su tamaño pequeño hace viable entrenar y evaluar en CPU.

Limitaciones

- Solo 467 archivos y un solo lenguaje.
- Los problemas son simples, así que los resultados pueden no generalizar a código más complejo.
- Hay desbalance: unos 355 positivos contra 105 negativos.



Descripción del modelo



Tipo de modelo

Sistema híbrido de similitud/clasificación binaria (plagio vs. no plagio) con dos componentes y un combinador

Analizador semántico (AST)

- Se parsea el código con tree-sitter.
- Se normaliza el árbol: se eliminan comentarios, se renombran identificadores y se canonizan los literales.
- Se comparan los árboles con distancia de edición (zss).
- Detecta cambios de nombres y de formato.

Modelo neuronal siamés

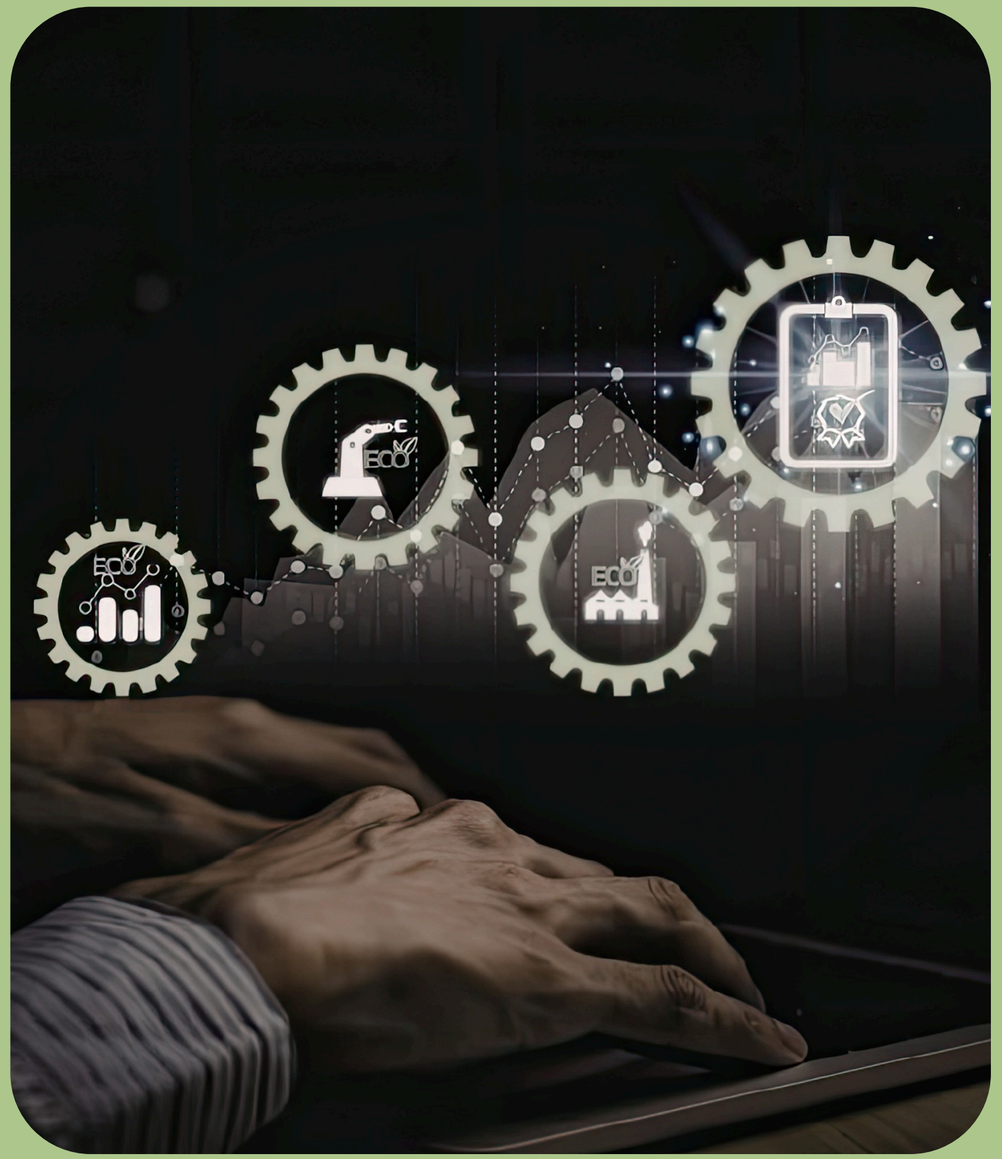
- Parte de un codificador preentrenado, microsoft/unixcoder-base.
- Compara los embeddings de ambos códigos.
- Detecta reescrituras que mantienen la misma lógica.

Combinador

- Fusiona ambos puntajes en un valor final en [0.0, 1.0] y genera un reporte legible con las regiones coincidentes.

Herramientas y lenguajes

Área	Herramientas
Lenguaje	Python 3.12
Deep learning	PyTorch (CPU), Hugging Face Transformers
Análisis de código	tree-sitter, zss
Interfaz	Typer (CLI: compare, scan, train, evaluate)
Despliegue	Docker / Docker Compose, uv, Makefile



Pasos clave del desarrollo

Preparación de datos y Preprocesamiento

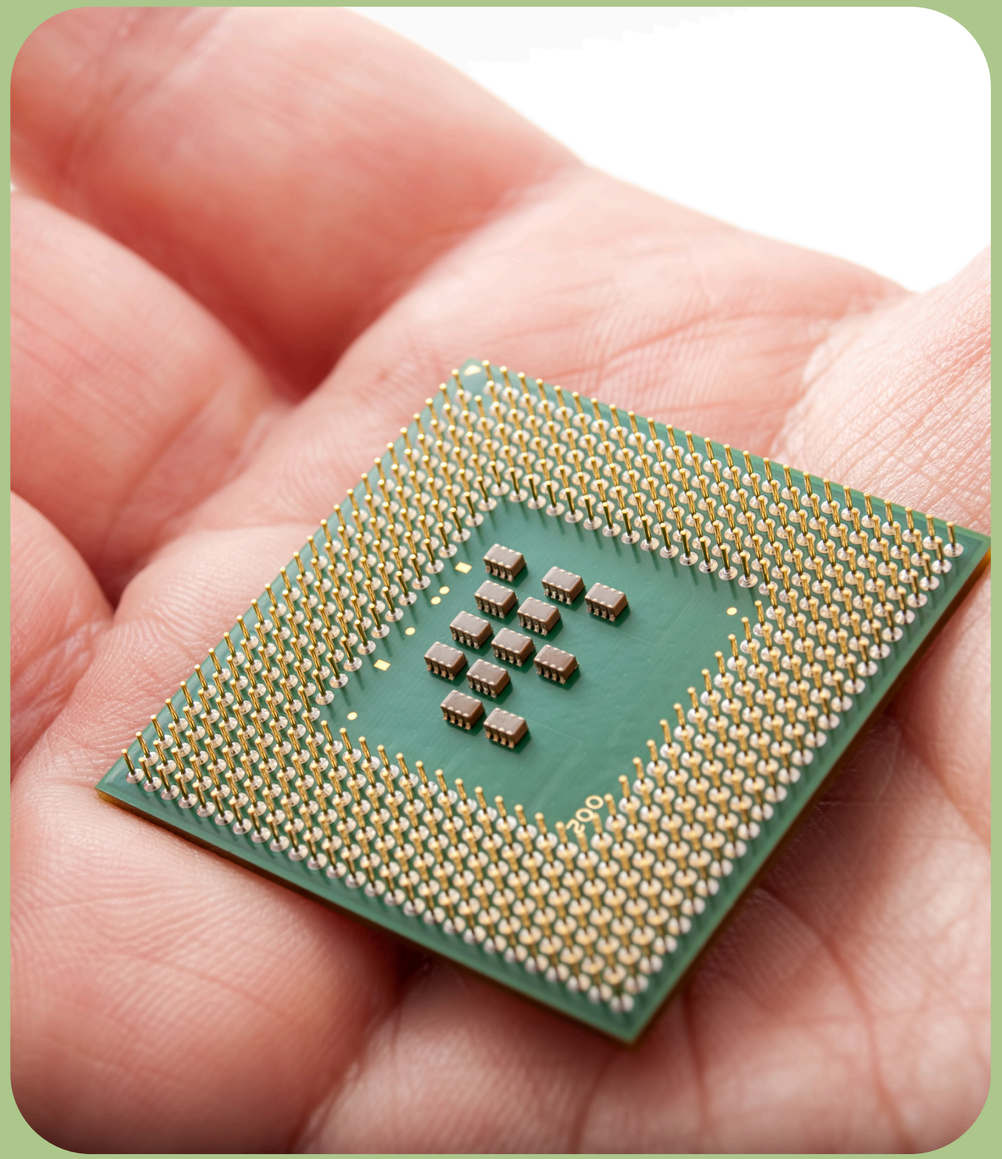
- Descarga y procesamiento, generación de pares positivos y negativos, y división por problema.
- Parseo a AST y normalización (sin comentarios, con identificadores y literales canónicos).

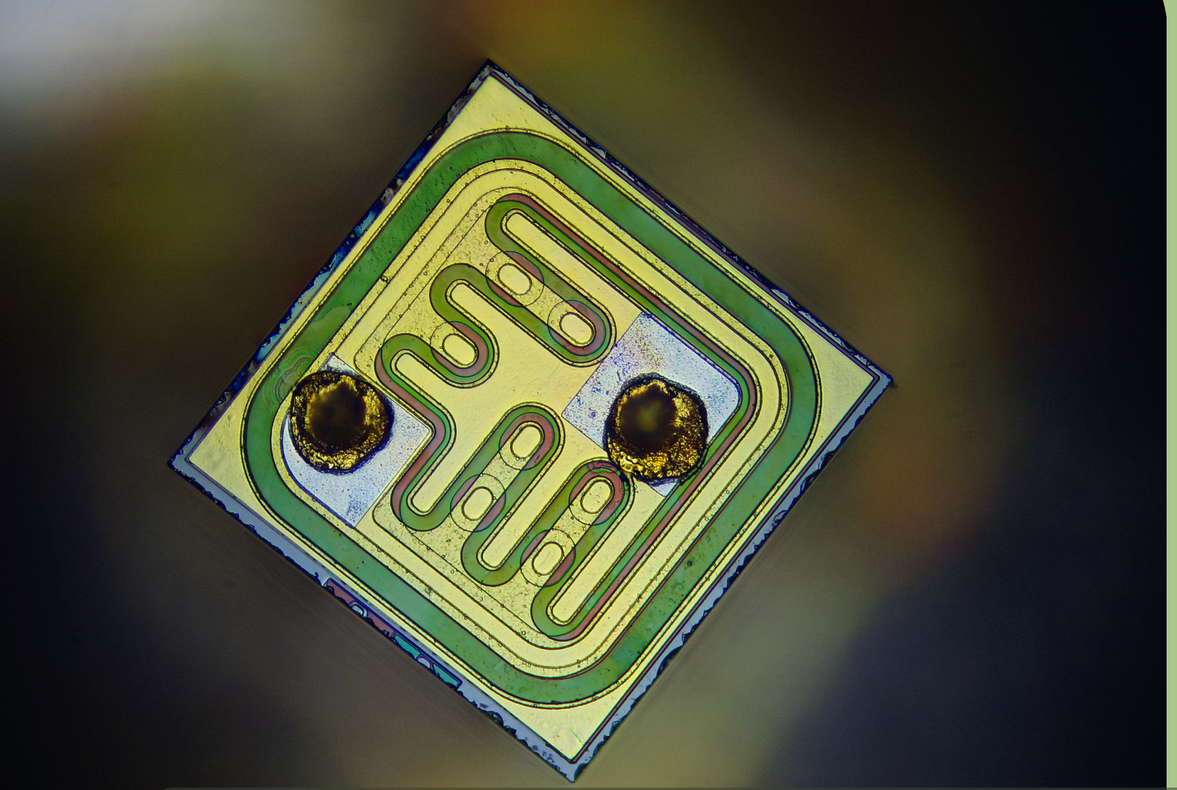
Entrenamiento y Validación y evaluación

- Red siamesa sobre el codificador preentrenado, configurada para ser factible en CPU, con semillas fijas para reproducibilidad.
- Evaluación en la partición de prueba, con resultados guardados en docs/results/.

Combinación de puntajes y Reporte e interfaz

- Fusión de ambos puntajes con umbrales configurables.
- Salida legible o JSON mediante la CLI.





Resultados

1 En Proceso

Modern technology ecosystems are built on interconnected digital platforms, intelligent infrastructure, and scalable systems that support innovation, collaboration, and efficient operations across industries while enabling continuous technological advancement in a rapidly evolving global environment.